

Unit-05 Three-Dimensional Graphics

5.1 3D Object Representation: Polygon Table

- 3D objects are represented using polygons, primarily triangles or quadrilaterals.
- **Polygon Table** stores information about the vertices, edges, and faces of an object.
- Example structure:
 - **Vertex Table:** Stores (x, y, z) coordinates of each vertex.
 - **Edge Table:** Stores pairs of vertices forming edges.
 - **Polygon Table:** Stores polygons as sets of edges or vertices.
- **Advantages of Polygon Representation:**
 - Efficient for rendering
 - Easy to manipulate and transform
 - Compatible with modern graphics hardware

5.2 Projection: Definition and Types

- **Projection:** Transformation of 3D objects onto a 2D plane.
- **Types of Projection:**
 - **Parallel Projection:** Preserves parallel lines, does not produce perspective distortion.
 - Orthographic Projection (Front, Top, Side views)
 - Oblique Projection (Cabinet, Cavalier projections)
 - **Perspective Projection:** Mimics human vision, objects appear smaller at a distance.
 - One-point, Two-point, and Three-point perspective
- **Applications of Projection:**
 - Used in engineering and architectural drawings
 - Essential for 3D modeling and rendering

Projection Matrices

- **Parallel Projection Matrix (Orthographic):**

$$P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Perspective Projection Matrix:**

$$P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

(where d is the distance from the viewer)

5.3 Visible Surface Detection Methods

- **Why is it needed?:** To determine which surfaces of 3D objects are visible and which are hidden.
- **Object Space Methods:**
 - **Depth Sorting (Painter's Algorithm):** Sorts polygons based on depth and draws from back to front.
- **Image Space Methods:**
 - **Z-Buffer Method:** Stores depth values for each pixel and updates only if a new surface is closer.
 - **A-Buffer Method:** Advanced version of Z-buffer storing transparency information.
 - **Scanline Method:** Processes one scanline at a time, determining visible surfaces.
- **Comparison of Methods:**
 - Z-Buffer: Simple but memory-intensive
 - A-Buffer: Handles transparency better
 - Scanline: Efficient for rendering complex scenes

5.4 Basic Illumination Model

- Determines how light interacts with surfaces.
1. **Ambient Light:** Uniform light affecting all objects equally.
 2. **Diffuse Reflection:** Light scattered equally in all directions.
 3. **Specular Highlights:** Reflection creating a bright spot on a surface.

Illumination Formula:

$$I = I_a K_a + I_d K_d (N \cdot L) + I_s K_s (R \cdot V)^n$$

Where:

- I_a, I_d, I_s = Intensities of ambient, diffuse, and specular light
- K_a, K_d, K_s = Material reflection coefficients
- NN = Surface normal, LL = Light vector, RR = Reflection vector, VV = View vector
- **Applications:** Used in 3D games, animation, and rendering realistic images.

5.5 Polygon Rendering Methods

1. **Constant Shading (Flat Shading):** Single color for the entire polygon.
2. **Gouraud Shading:** Interpolates vertex colors across the polygon.
3. **Phong Shading:** Interpolates surface normals and calculates illumination per pixel.

Example (Phong Shading in OpenGL)

```
// Set up Phong shading in OpenGL
void main() {
    vec3 norm = normalize(Normal);
    vec3 lightDir = normalize(LightPos - FragPos);
    float diff = max(dot(norm, lightDir), 0.0);
    vec3 diffuse = diff * LightColor;
    FragColor = vec4(diffuse, 1.0);
}
```

- **Comparison:**
 - Flat Shading: Fast but unrealistic
 - Gouraud Shading: Better, but may have color inconsistencies
 - Phong Shading: Most realistic, but computationally expensive

5.6 RGB, HSV, and CMYK Color Models

- **RGB (Red, Green, Blue):** Additive model used in screens.
- **HSV (Hue, Saturation, Value):** Better for human perception.
- **CMYK (Cyan, Magenta, Yellow, Black):** Used in printing.

Conversion Example (RGB to HSV in Python)

```
import colorsys
r, g, b = 0.5, 0.2, 0.8
h, s, v = colorsys.rgb_to_hsv(r, g, b)
print(h, s, v)
```

Additional Concepts for Better Understanding

- **Clipping in 3D Graphics:** Removing parts of objects that are outside the view.
- **Transformation in 3D Graphics:**
 - **Translation:** Moving an object in 3D space
 - **Rotation:** Rotating an object around an axis
 - **Scaling:** Changing the size of an object
- **Ray Tracing vs Rasterization:**
 - **Ray Tracing:** Realistic, used in high-end graphics
 - **Rasterization:** Faster, used in real-time applications like video games